Growth of Functions + Notasi Asymptotic

MATERI

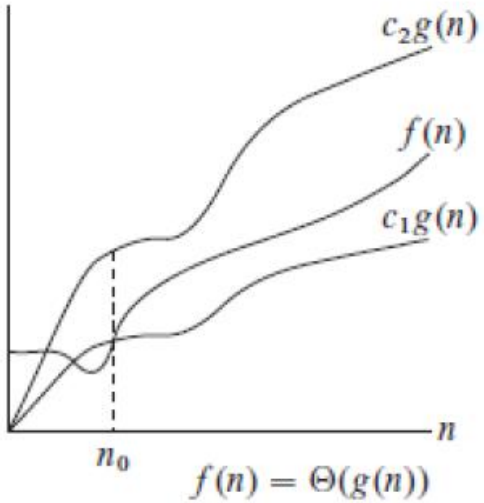
1. Pengantar Kompleksitas Algoritma.
   - Peran Algoritma dalam Komputer
   - Pengantar DAA.
   - Growth of Functions + Notasi Asymptotic
2. Analisa Kompleksitas:
   - Rekursif : Fibonacci
   - Sorting: **Heap sort**, Selection Sort, Insertion Sort, Bubble Sort, Shell Sort, Merge Sort, Quick Sort
   - Searching: Sequential Search, Fibonacci Search, Interpolation Search, Binary Search, Fibonacci Search, Depth First Search (DFS), dan Breadth First Search (BFS).
3. Non-Polynomial Completeness (NPC) dan Non-Polynomial Hardness (NPH):
   - Greedy Algorithm
   - Dynamic Programming
     - Knapsack Problem
   - Travelling Salesman Problem

PertumbuhanFungsi
(Growth of Function)

Fungsi running time terdiri dari :

- Ukuran input : n
- Konstanta perkalian : c_1, c_2
- Suku : lower-order s/d highest-order: n, n^2

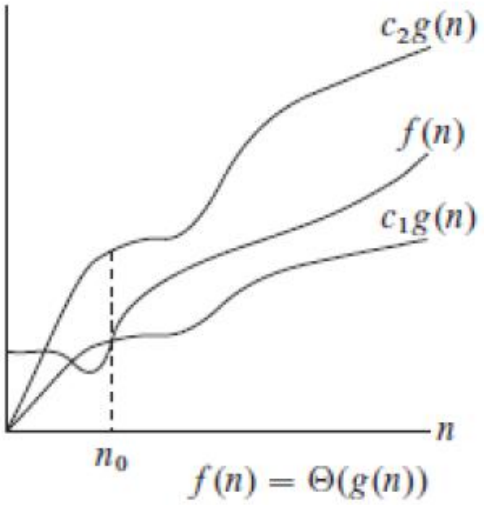
Misal : f(n) = an^2 + bn +c
g(n) = n^2



- Pertumbuhan (growth) dari running time suatu algoritma mengkarakterisasikan efisiensi dari algoritma.
- Semakin cepat pertumbuhan, semakin berkurang efisiensi.
- Saat ukuran input n menjadi cukup besar :
  - konstanta perkalian dan suku 'lower-order' dari fungsi running time eksak didominasi oleh efek dari ukuran input itu sendiri.
  - Hanya orde pertumbuhan dari running time yang relevan

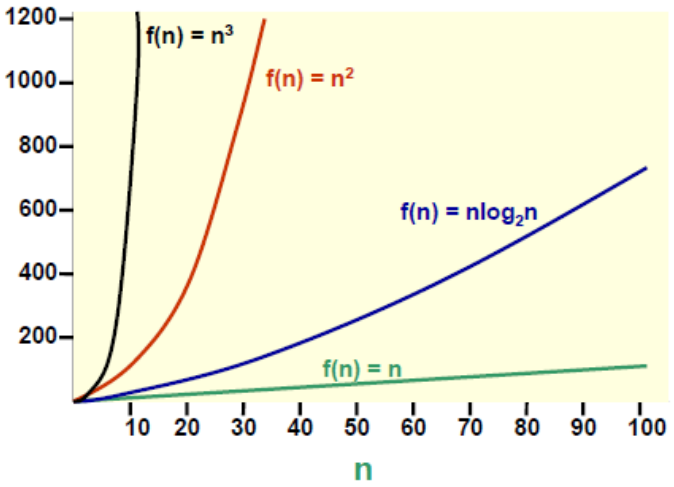
4
3,4
3

Pertumbuhan Fungsi



- Saat ukuran input cukup besar untuk membuat orde pertumbuhan dari running time menjadi relevan:
- dilakukan study efisiensi asymptotic dari algoritma.
- > untuk memperhatikan : bagaimana running time dari algoritma bertambah dengan ukuran input **n** , saat **n** bertambah tanpa batas.
- Algoritma yang secara asysmptotic lebih efisien akan menjadi pilihan terbaik, tetapi dengan ukuran input yang sangat kecil.

Pertumbuhan Fungsi



- Nilai pendekatan dari fungsi yang bervariasi dari n.
- kecepatan pertumbuhan : menggunakan fakta : $2^{10}=10^{3.0103}$

fungsi

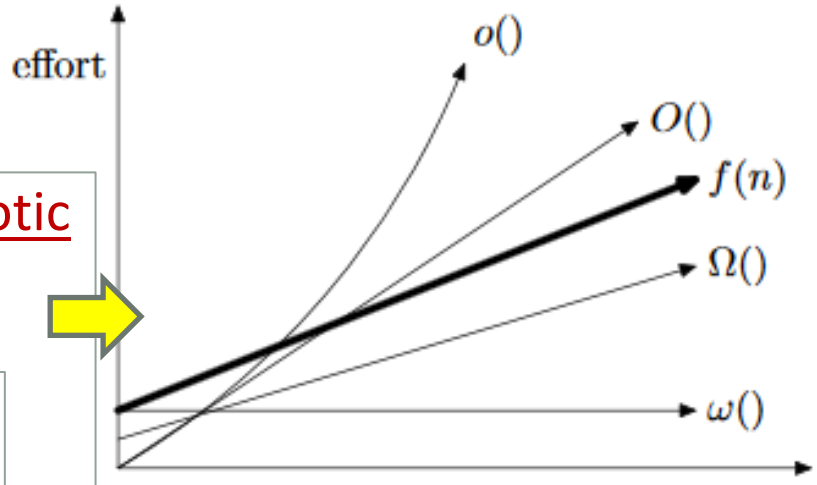
$\lg n$	3	6	9	13	16	19
n	10	10^2	10^3	10^4	10^5	10^6
$n \lg n$	30	664	9965	10^5	10^6	10^7
n^2	10^2	10^4	10^6	10^8	10^{10}	10^{12}
n^3	10^3	10^6	10^9	10^{12}	10^{15}	10^{18}
2^n	10^3	10^{30}	10^{301}	10^{3010}	10^{30103}	10^{301030}

Notasi Asymptotic

- Digunakan untuk : Study efisiensi asymptotic dari algoritma

1. Notasi-Θ (big theta)
2. Notasi-O (big o)
3. Notasi-Ω (big omega)
4. Notasi-o (little-o)
5. Notasi ω (little omega)

5
4
3,4
3
2



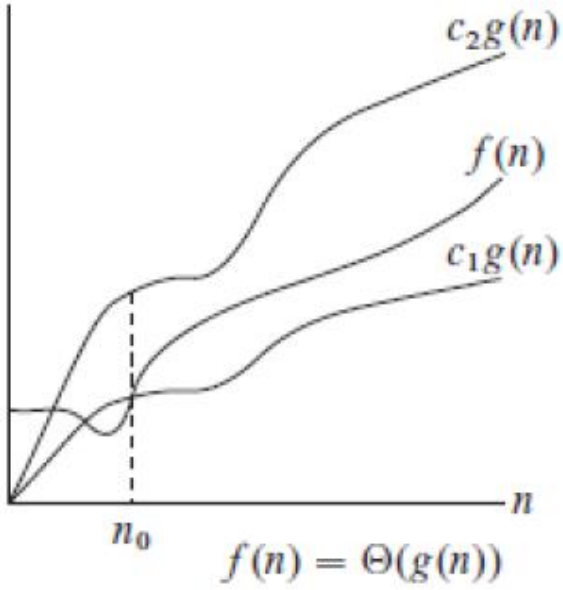
O is like ≤ : f(n) = O(g(n)) means f(n) ≤ cg(n)

Ω is like ≥ : f(n) = Ω(g(n)) means f(n) ≥ cg(n)

Θ is like = : f(n) = Θ(g(n)) ⇔ g(n) = Θ(f(n))

o is like < : f(n) = o(g(n)) means f(n) < cg(n)

ω is like > : f(n) = ω(g(n)) means f(n) > cg(n)



Misal : f(n) = ax^2 + bx +c
g(n) = x^2

Definition	? $c > 0$	? $n_0 \geq 1$	$f(n)$? $c \cdot g(n)$
$O()$	$\exists$	$\exists$	$\leq$
$o()$	$\forall$	$\exists$	$<$
$\Omega()$	$\exists$	$\exists$	$\geq$
$\omega()$	$\forall$	$\exists$	$>$

1- Notasi-Θ (big Theta)

- Untuk suatu fungsi g(n), Θ(g(n)) adalah himpunan fungsi f(n), yang didefinisikan sbb :

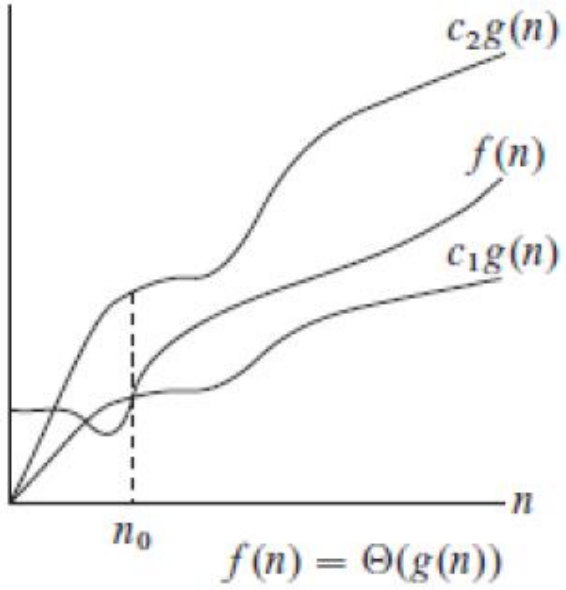
$3 \leq 3{,}4 \leq 4$-

$\Theta(g(n))=\{ f(n) :$ ada konstanta positif c_1, c_2 dan n_0 dimana $0 \leq c_1\, g(n) \leq f(n) \leq c_2\, g(n)$ untuk semua $n \geq n_0 \}$

- Untuk mengindikasikan bahwa f(n) adalah anggota dari Θ(g(n)), ditulis:

  f(n)ϵ Θ(g(n)) atau f(n) = Θ(g(n))

- g(n) secara asymptotic batas- untuk f(n)



Misal : f(n) = ax^2 + bx +c
g(n) = x^2

1- Notasi-Θ --Contoh

f(n) = Θ(g(n))

- Contoh : tunjukkan bahwa $\frac{1}{2}n^2 - 3n = \Theta(n^2)$

c1 ≤ 2 ≤ c2

- Harus ditentukan konstanta positif c_1, c_2 dan n_0 Sehingga

$$c_1 n^2 \le \frac{1}{2}n^2 - 3n \le c_2 n^2 \quad \text{for all } n \ge n_0$$

$0 \le c_1\, g(n) \le f(n) \le c_2\, g(n)$

- Pembagian dengan n^2 menghasilkan

$$c_1 \le \frac{1}{2} - \frac{3}{n} \le c_2$$

Nilai c1-c2:
>= 0 s/d 1/2

n = 1/2/3/4..

½ - x >=0

n=7→.3/7
7/14- 6/14

¾
1
½
2

c1 ≤ ½ − 0 ≤ c2

- Ketidaksamaan di sebelah kanan, hanya untuk semua nilai n ≥ 1 dengan memilih konstanta $c_2 \ge ½$
- Ketidaksamaan di sebelah kiri , hanya untuk semua nilai n ≥ 7 dengan memilih konstanta $c_1 \le 1/14$.
- Dengan memilih $c_1 = 1/14$, $c_2 = ½$ dan $n_0 = 7$, dapat diverifikasi bahwa :

$$\frac{1}{2}n^2 - 3n = \Theta(n^2)$$

- CAT: ada pilihan lain untuk konstanta, tetapi yang paling penting, adalah : ada beberapa pilihan yang benar

1- Notasi-Θ -- Contoh -2

- Tunjukkan bahwa $6n^3 \neq \Theta(n^2)$

- Untuk tujuan kontradiksi, dimana ada c_2 dan n_0 sehingga
- $6n^3 \leq c_2 n^2$ untuk semua $n \geq n_0$
- Pembagian dengan n^2 menghasilkan $n \leq c_2/6$. dimana tidak dapat dipenuhi dengan sembarang n yang besar, karena c_2 konstan

1-Notasi-Θ

- Secara intuitif, suku ‘orde rendah’ dari fungsi positif asymptotic dapat diabaikan dalam penentuan asymptotic batas, karena suku tersebut tidak signifikan untuk n berukuran besar.
- Penambahan kecil dari suku ‘orde tertinggi’ -- cukup untuk mendominasi suku ‘orde rendah’ :
  - Setting c_1 ke nilai yang sedikit lebih kecil dari koefisien dari suku orde tertinggi’ dan
  - setting c_2 ke nilai yang sedikit lebih besar
  - → membolehkan ketidaksamaan pada definisi dari notasi-theta dipenuhi

- Contoh : untuk fungsi kuadrat apapun, $f(n) = an^2 + bn + c$. Dimana a, b dan c adalah konstanta dan $a > 0$, didapat :
- $f(n) = \Theta(n^2)$
- >> Buktikan !

1-Notasi-Θ

- Secara umum, untuk semua polinomial

$$p(n) = \sum_{i=0}^{d} a_i n^i$$

- dimana a_i adalah konstanta dan $a_d > 0$

- $P(n) = \Theta(n^d)$

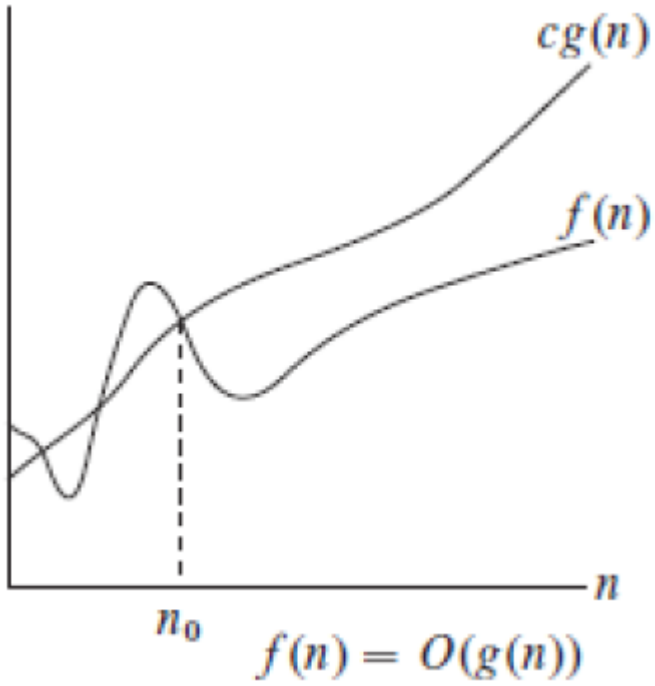
(fn)=3 = 3.n^0

- Untuk konstanta adalah polinomial derajat-0, dapat diekspresikan fungsi konstanta : $\Theta(n^0)$ atau $\Theta(1)$
- Kasus terburuk running time dari insertion sort adalah $\Theta(n^2)$.
- Jika input telah terurut, insertion sort run dalam waktu $\Theta(n)$

2-Notasi-O (big-O)

- Notasi-theta secara asymptotic membatasi suatu fungsi dari upper bound dan lower bound.
- Jika hanya mempunyai asymptotic upper bound, digunakan notasi-O

$O(g(n))=\{f(n)$: ada konstanta positif c dan n_0 dimana $0 \leq f(n) \leq c\, g(n)$ untuk semua $n \geq n_0\}$



2-Notasi-O

- f(n) = Θ(g(n)) berarti bahwa f(n) = O(g(n)), dimana notasi-Θ adalah bagian dari notasi-O

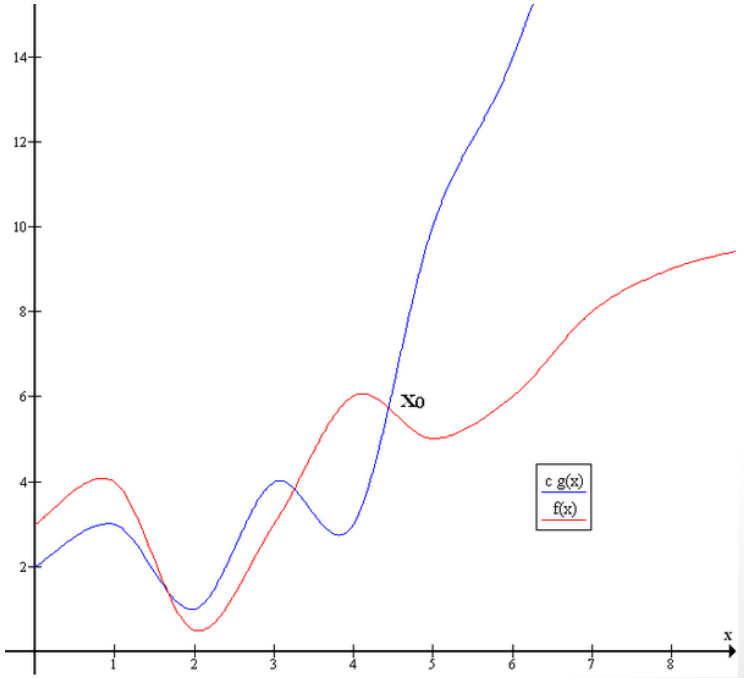
$$\Theta(g(n)) \subseteq O(g(n))$$

- Contoh

$$\frac{1}{2}n^2 - 3n = O(n^2)$$

$$an + b = O(n^2) \qquad a > 0$$

- Example of Big O notation: $f(x) \in O(g(x))$ as there exists $c > 0$ (e.g., $c = 1$) and x_0 (e.g., $x_0 = 5$) such that $f(x) \leq cg(x)$ whenever $x \geq x_0$.

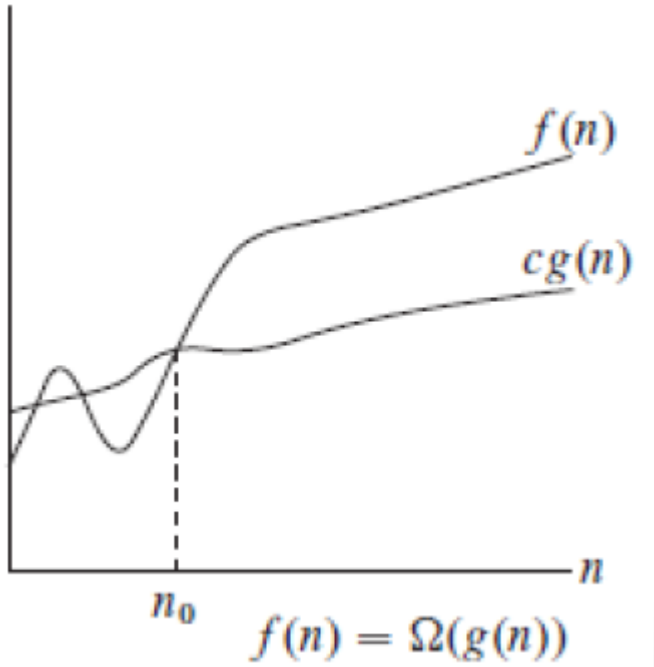


3-Notasi Ω (big omega)

- Menyediakan asymptotic lower bound

  $\Omega(g(n))=\{f(n)$: ada konstanta positif c dan n_0 dimana $0 \leq c\ g(n) \leq f(n)$ untuk semua $n \geq n_0\}$

- Contoh :
  - $an^2 + bn + c = \Omega(n^2)$ $a > 0$
  - $7n^2 = \Omega(n)$



Teorema

- Untuk dua fungsi f(n) dan g(n), kita mempunyai

  $f(n) = \Theta(g(n))$ jika dan hanya jika

  $f(n) = O(g(n))$ dan

  $f(n) = \Omega(g(n))$
- Teorema ini biasa digunakan untuk membuktikan batas asymptotic dari asymptotic upper bound dan asymptotic lower bound

Latihan

- $\sqrt{n} = \Omega(\lg n)$

  $c = ?$

  $n_0 = ?$

- $\frac{1}{2}n^2 - 2n = \Theta(n^2)$

  $c_1 = ?$

  $c_2 = ?$

  $n_0 = ?$

4-Notasi-o (little-oh)

- Asymptotic upper bound yang disediakan oleh notasi-o boleh atau tidak boleh menjadi batas asymptotic.
- Batas $2n^2 = o(n^2)$ adalah batas asymptotic, tetapi
- Batas $2n = o(n^2)$ tidak
- Notasi-o digunakan untuk menunjukkan suatu upper bound yang tidak batas asymptotic

$o(g(n))=\{f(n)$: untuk konstanta positif $c > 0$, ada konstanta $n_0 > 0$, dimana $0 \leq f(n) \leq c\ g(n)$ untuk semua $n \geq n_0\}$

- $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$

Pangkat n →di f(n) lebih kecil dari pangkat di g(n)

5-Notasi-ω (little-omega)

- Notasi-ω digunakan untuk menunjukkan bahwa lower bound bukan batas asymptotic.
- f(n)= ω(g(n)) jika dan hanya jika g(n)=o(f(n))

ω(g(n))={f(n): untuk konstanta positif c > 0, ada konstanta $n_0 > 0$, dimana $0 \leq c\ g(n) \leq f(n)$ untuk semua $n \geq n_0$}

$$\lim_{n\to\infty} \frac{g(n)}{f(n)} = 0$$

contoh

$$\lim_{x\to\infty} \frac{2x+3}{3x^2+5x-2} = \lim_{x\to\infty} \frac{\frac{2x}{x^2}+\frac{3}{x^2}}{\frac{3x^2}{x^2}+\frac{5x}{x^2}-\frac{2}{x^2}}$$

$$= \lim_{x\to\infty} \frac{\frac{2}{x}+\frac{3}{x^2}}{3+\frac{5}{x}-\frac{2}{x^2}} = \frac{\frac{2}{\infty}+\frac{3}{\infty}}{3+\frac{5}{\infty}-\frac{2}{\infty}}$$

Sehingga akan di peroleh,

$$\lim_{x\to\infty} \frac{2x+3}{3x^2+5x-2} = \frac{0+0}{3+0+0} = \frac{0}{3} = 0$$

Pangkat n →di g(n) lebih kecil dari pangkat di f(n)

Logaritma

$$\lg n = \log_2 n \quad \text{(binary logarithm)}$$

$$\ln n = \log_e n \quad \text{(natural logarithm)}$$

$$\lg^k n = (\lg n)^k \quad \text{(exponentiation)}$$

$$\lg \lg n = \lg(\lg n) \quad \text{(composition)}$$

$$\lg n + k = (\lg n) + k$$

Logaritma

- Untuk semua bilangan riil a > 0, b > 0, c > 0 dan n

$$a = b^{\log_b a}$$

$$\log_c (ab) = \log_c a + \log_c b$$

$$\log_b a^n = n \log_b a$$

$$\log_b a = \frac{\log_c a}{\log_c b}$$

$$\log_b (1/a) = -\log_b a$$

$$\log_b a = \frac{1}{\log_a b}$$

$$a^{\log_b c} = c^{\log_b a}$$

Fungsi Logaritma Iterasi

$$f^{(i)}(n) = \begin{cases} n & \text{if } i = 0 \\ f(f^{(i-1)}(n)) & \text{if } i > 0 \end{cases}$$

$$\lg^* n = \min\{i \geq 0 : \lg^{(i)} n \leq 1\}$$

Logaritma iterasi adalah fungsi yang tumbuh sangat lambat

$$\lg^* 2 = 1$$

$$\lg^* 4 = 2$$

$$\lg^* 16 = 3$$

$$\lg^* 65536 = 4$$

$$\lg^*(2^{65536}) = 5$$